



NT533-Report-Group 5 - Đồ án môn học làm về CICD pipeline và IaC

Mang may tinh (University of Information Technology)



messages.pdf_cover_qr_code_label

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

Trường Đại Học Công Nghệ Thông Tin



Đồ án môn học

CI/CD with Infrastructure as Code

Hệ tính toán phân bố - NT533.Q13

Nhóm 5

GVHD: Huỳnh Văn Đặng

Sinh viên thực hiện:

Huỳnh Lâm Tuấn Phong-23521162

Trần Minh Nhật-23521099

Mục lục

Bảng phân công.....	3
1. Nội dung báo cáo.....	4
1.1 Giới thiệu chung.....	4
1.2 Mục tiêu đề tài.....	4
1.3 Phạm vi thực hiện.....	5
2. Nền tảng lý thuyết.....	6
2.1 Tổng quan.....	6
2.1.1 DevOps.....	6
2.1.2 CI/CD.....	7
2.1.3 IaC.....	7
2.2 Các công cụ sử dụng.....	8
2.2.1 Terraform.....	8
2.2.2 Docker.....	10
2.2.3 Kubernetes light (K3s).....	11
2.2.4 Github Actions.....	12
3. Kịch bản thử nghiệm.....	14
3.1 Deploy K3s Cluster thông qua Terraform.....	14
3.1.1 Mục tiêu.....	14
3.1.2 Thực hiện.....	14
3.1.3 Kết quả.....	18
3.2 Tự động hóa triển khai khi thay đổi mã nguồn.....	19
3.2.1 Mục tiêu.....	19
3.2.2 Thực hiện.....	19
3.2.2.1 Cài đặt Self-hosted runner.....	19
3.2.2.2 CI/CD workflows.....	21
3.2.3 Kết quả.....	22
3.3 Rollback khi production bị lỗi hoặc khi deploy lỗi.....	24
3.3.1 Mục tiêu.....	24
3.3.2 Thực hiện.....	24
3.3.2.1 Theo dõi rollout và xử lý lỗi triển khai.....	24
3.3.2.2 Rollback tự động khi triển khai thất bại.....	24
3.3.3 Kết quả.....	25
3.4 High Availability & Load Balancer bằng thuật toán chia tải Round Robin.....	27
3.4.1 Mục tiêu.....	27
3.4.2 Thực hiện.....	28
3.4.2.1 Traefik.....	28
3.4.2.2 Cách thực hiện.....	28

3.4.3 Kết quả.....	29
3.4.3.1 Load Balance.....	29
3.4.3.2 High Availability.....	30
3.5 Video thực hiện.....	32
4. Tổng kết.....	33
5. Tài liệu tham khảo.....	34

Bảng phân công

Họ tên	MSSV	Công việc	Hoàn thành
Trần Minh Nhật	23521099	Thực hiện đồ án, quay demo, thuyết trình, viết báo cáo	100%
Huỳnh Lâm Tuấn Phong	23521162	Thực hiện đồ án, làm side, thuyết trình, viết báo cáo	100%

1. Nội dung báo cáo

1.1 Giới thiệu chung

Việc triển khai và quản lý hệ thống hạ tầng phần cứng và phần mềm theo cách thủ công truyền thống đang bộc lộ nhiều hạn chế nghiêm trọng trong bối cảnh công nghệ thông tin ngày càng phát triển và trở thành một phần quan trọng của mọi hoạt động kinh doanh. Những hạn chế này bao gồm thời gian tốn kém cho các tác vụ lặp đi lặp lại, khó kiểm soát và theo dõi các thay đổi trong hệ thống, khả năng lỗi do yếu tố con người cao và đặc biệt là gặp khó khăn lớn khi cần mở rộng quy mô hệ thống để đáp ứng nhu cầu.

Để giải quyết triệt để những vấn đề trên, các mô hình Infrastructure as Code (IaC) và Continuous Integration/Continuous Deployment (CI/CD) đã ra đời và nhanh chóng trở thành xu hướng tất yếu trong ngành công nghệ.

IaC cho phép định nghĩa toàn bộ hạ tầng dưới dạng mã nguồn có thể quản lý phiên bản, trong khi CI/CD giúp tự động hóa toàn bộ quy trình từ việc tích hợp mã nguồn, kiểm thử đến triển khai ứng dụng. Sự kết hợp này không chỉ giảm thiểu sai sót mà còn tăng tốc độ phát triển và triển khai sản phẩm, đáp ứng yêu cầu cạnh tranh gay gắt của thị trường hiện nay.

Đề tài “Triển khai hạ tầng bằng Terraform và CI/CD bằng GitHub Actions” được lựa chọn nhằm nghiên cứu và áp dụng các công cụ hiện đại trong quy trình DevOps, giúp xây dựng một hệ thống có khả năng triển khai tự động, dễ mở rộng và đảm bảo tính nhất quán giữa các môi trường (development – staging – production).

Đây không chỉ là một hướng nghiên cứu mang tính thực tiễn cao, mà còn giúp chúng ta hiểu rõ hơn về cách thức kết hợp giữa tự động hóa hạ tầng (IaC) và tự động hóa triển khai phần mềm (CI/CD) trong các dự án phần mềm hiện đại.

1.2 Mục tiêu đề tài

1. Xây dựng quy trình CI/CD tự động sử dụng GitHub Actions:

Đề tài tập trung phát triển một quy trình tự động hóa phát triển và triển khai phần mềm, trong đó toàn bộ các giai đoạn như kiểm thử, biên dịch, đóng gói và phát hành ứng dụng đều được vận hành tự động thông qua GitHub Actions.

Công cụ này được lựa chọn vì nó cho phép kích hoạt các quy trình dựa trên sự kiện thay đổi mã nguồn, đảm bảo việc tích hợp và triển khai liên tục diễn ra đồng bộ và nhất quán. Hệ thống tự động thực hiện việc kiểm tra mã, xây dựng web, tạo image Docker, và tải lên kho chứa container để sẵn sàng cho bước triển khai.

2. Ứng dụng Infrastructure as Code (IaC):

Mã nguồn quản lý toàn bộ hạ tầng triển khai bằng cách sử dụng các công cụ IaC như Terraform hoặc Ansible. Cách tiếp cận này đảm bảo tính tự động, có thể tái sử dụng và dễ mở rộng đồng thời duy trì tính nhất quán giữa các môi trường (phát triển, kiểm tra, đưa ra sản phẩm).

→ Ngoài ra, IaC cho phép kiểm soát phiên bản hạ tầng bao gồm mã phần mềm, làm cho việc mở rộng và khôi phục hệ thống thuận tiện và nhanh chóng.

3. Tích hợp Docker và K3s trong mô hình container hóa:

Để đảm bảo tính di động và khả năng chạy đồng nhất trên mọi nền tảng, sản phẩm được đóng gói dưới dạng container thông qua Docker.

K3s cung cấp đầy đủ các tính năng quản lý container, giám sát và mở rộng quy mô, mô tả cách các doanh nghiệp sử dụng hệ thống Kubernetes thực tế.

Việc triển khai trên K3s, một phiên bản nhẹ của Kubernetes, giúp mô phỏng môi trường triển khai thực tế của các hệ thống phân tán và phù hợp với các điều kiện tài nguyên giới hạn trong môi trường thử nghiệm hoặc học tập.

4. Thiết lập cơ chế kiểm soát lỗi và rollback tự động:

Việc phát triển cơ chế phát hiện lỗi và khôi phục tự động khi gặp sự cố trong quá trình triển khai là một trong mục tiêu chính của đề tài. Nếu có lỗi trong bản phát hành mới, hệ thống có thể tự động quay trở lại phiên bản ổn định trước đó bằng cách sử dụng các công cụ quản lý phiên bản triển khai như Helm hoặc Argo CD.

→ Điều này làm tăng độ tin cậy và khả năng phục hồi của hệ thống đồng thời giảm thiểu việc bị gián đoạn dịch vụ.

5. Đánh giá hiệu quả của hệ thống CI/CD kết hợp IaC:

Các tiêu chí đánh giá bao gồm: thời gian triển khai, tỷ lệ lỗi phát sinh, mức độ tự động hóa, và độ ổn định của hệ thống sau khi triển khai. Kết quả so sánh này giúp chứng minh tính ưu việt của việc áp dụng CI/CD và IaC trong việc nâng cao hiệu suất làm việc của nhóm phát triển, giảm thiểu sai sót con người và tăng tính linh hoạt của hạ tầng công nghệ.

1.3 Phạm vi thực hiện

Công nghệ sử dụng:

- **GitHub Actions:** Dùng để thiết lập pipeline CI/CD tự động.

- **Docker:** Đóng gói ứng dụng thành các container có thể triển khai độc lập.
- **K3s:** Phiên bản nhẹ của Kubernetes dùng để triển khai và quản lý container trên môi trường mô phỏng cụm cluster.
- **Terraform hoặc YAML manifest:** Dùng để định nghĩa và triển khai hạ tầng theo mô hình Infrastructure as Code.
- **GitHub Repository:** Lưu trữ mã nguồn, quản lý phiên bản và kích hoạt workflow CI/CD.

Môi trường triển khai:

- Hệ thống được mô phỏng trên cụm K3s bao gồm **một node Master** và **một hoặc nhiều node Worker**, chạy trên nền máy ảo hoặc máy vật lý.
- Toàn bộ quá trình build, deploy, và rollback được thực hiện tự động thông qua pipeline CI/CD.

Giới hạn của đề án:

- Tập trung vào thiết kế và triển khai quy trình CI/CD với khả năng rollback khi xảy ra lỗi.
- Không đi sâu vào tối ưu hiệu năng hoặc triển khai trên môi trường điện toán đám mây quy mô lớn.

2. Nền tảng lý thuyết

2.1 Tổng quan

2.1.1 DevOps

Mô hình DevOps là một phương pháp tiếp cận trong phát triển phần mềm hiện đại, kết hợp chặt chẽ giữa hai lĩnh vực Phát triển phần mềm (Development) và Vận hành hệ thống (Operations) nhằm hình thành một chu trình làm việc thống nhất, liên tục và hiệu quả. DevOps hướng đến mục tiêu xóa bỏ ranh giới giữa nhóm phát triển và nhóm vận hành, tạo ra sự cộng tác xuyên suốt trong toàn bộ vòng đời của sản phẩm – từ giai đoạn thiết kế, kiểm thử, triển khai cho đến giám sát và bảo trì.

Cốt lõi của DevOps nằm ở việc tự động hóa các quy trình trong phát triển và triển khai phần mềm, giúp rút ngắn thời gian phát hành, giảm thiểu sai sót do thao tác thủ công, đồng thời nâng cao khả năng phản ứng của hệ thống trước các thay đổi. Thông qua việc áp dụng các công cụ và phương pháp DevOps, doanh nghiệp có thể tăng tốc độ phát triển, phát hiện và khắc phục lỗi sớm hơn, cũng như đảm bảo tính ổn định và khả năng mở rộng của hạ tầng công nghệ thông tin.

Việc triển khai DevOps không chỉ mang lại lợi ích về mặt kỹ thuật mà còn thay đổi văn hóa tổ chức, hướng đến một mô hình vận hành linh hoạt, minh bạch và liên tục cải tiến. Đây cũng chính là nền tảng tư tưởng và kỹ thuật quan trọng để hiện thực hóa các mô hình Tích hợp – Triển khai liên tục (CI/CD) và Hạ tầng dưới dạng mã (Infrastructure as Code – IaC), góp phần nâng cao mức độ tự động hóa và tính nhất quán trong quản lý hệ thống phần mềm hiện đại.

2.1.2 CI/CD

CI/CD là viết tắt của Continuous Integration (Tích hợp liên tục) và Continuous Deployment/Delivery (Triển khai liên tục hoặc Phân phối liên tục). Đây là một trong những phương pháp cốt lõi của DevOps, nhằm tự động hóa toàn bộ quá trình phát triển, kiểm thử và triển khai phần mềm.

Mục tiêu của CI/CD là rút ngắn vòng đời phát triển phần mềm, giảm thiểu sai sót do con người, và đảm bảo các phiên bản ứng dụng mới được đưa đến người dùng một cách nhanh chóng, an toàn và ổn định.

- **Continuous Integration (CI):**

Là quá trình tự động tích hợp mã nguồn từ nhiều nhà phát triển vào một nhánh chính (main branch) thường xuyên, có thể là hàng ngày hoặc mỗi lần commit. Hệ thống CI sẽ tự động build, kiểm thử (unit test, integration test) và thông báo kết quả đến nhóm phát triển.

Mục tiêu của CI là phát hiện sớm lỗi tích hợp, giúp đảm bảo mã nguồn luôn trong trạng thái có thể build và triển khai được.

- **Continuous Delivery (CD):**

Là bước mở rộng của CI, trong đó ứng dụng sau khi vượt qua các bước kiểm thử sẽ được đóng gói, đẩy lên môi trường staging hoặc pre-production. Quá trình triển khai vẫn có thể được kích hoạt thủ công, nhưng đảm bảo mỗi phiên bản đều có thể được phát hành bất cứ lúc nào.

- **Continuous Deployment (CD):**

Là cấp độ cao hơn của Continuous Delivery, trong đó việc triển khai ứng dụng lên môi trường sản xuất được tự động hóa hoàn toàn sau khi vượt qua toàn bộ các giai đoạn kiểm thử.

Continuous Deployment giúp loại bỏ hoàn toàn rào cản giữa giai đoạn phát triển và vận hành, đảm bảo tốc độ phát hành nhanh và khả năng phản hồi thay đổi linh hoạt.

2.1.3 IaC

Infrastructure as Code (IaC) là mô hình quản lý và cung cấp hạ tầng thông qua mã nguồn thay vì thực hiện thủ công trên giao diện người dùng hoặc dòng lệnh. Toàn bộ cấu hình hệ thống như máy chủ, mạng, lưu trữ, container, và các dịch vụ phụ trợ đều được mô tả bằng các tệp cấu hình (ví dụ: YAML, JSON, HCL).

Với IaC, việc tạo lập, thay đổi hoặc tái tạo hạ tầng có thể được tự động hóa hoàn toàn, đảm bảo tính nhất quán, khả năng tái sử dụng và dễ dàng kiểm soát phiên bản.

Mô hình này là nền tảng quan trọng trong DevOps và Cloud Computing, giúp rút ngắn thời gian triển khai, giảm thiểu lỗi con người, và đảm bảo các môi trường phát triển, kiểm thử và sản xuất luôn đồng bộ với nhau.

IaC có thể được triển khai theo hai mô hình chính:

- **Declarative (khai báo):** Người dùng chỉ cần mô tả trạng thái cuối cùng mong muốn của hạ tầng, công cụ IaC sẽ tự động tính toán các bước cần thiết để đạt được trạng thái đó.
→ Ví dụ: Terraform, Ansible, CloudFormation.
- **Imperative (mệnh lệnh):** Người dùng mô tả chi tiết từng bước để tạo lập hoặc cấu hình hạ tầng. Mô hình này linh hoạt hơn nhưng khó bảo trì hơn khi quy mô lớn.
→ Ví dụ: Bash script, PowerShell, hoặc công cụ CLI truyền thống.

Lợi ích chính của IaC gồm:

- Tự động hóa việc thiết lập hạ tầng, tiết kiệm thời gian và công sức.
- Đảm bảo tính nhất quán giữa các môi trường (dev, test, production).
- Dễ dàng tái sử dụng, mở rộng hoặc khôi phục khi xảy ra sự cố.
- Theo dõi phiên bản hạ tầng như mã nguồn phần mềm.

IaC là nền tảng quan trọng trong mô hình DevOps hiện đại, cho phép đồng bộ giữa việc triển khai ứng dụng (Application Deployment) và hạ tầng (Infrastructure Deployment).

2.2 Các công cụ sử dụng

2.2.1 Terraform

Terraform là một công cụ Infrastructure as Code (IaC) mã nguồn mở do HashiCorp phát triển, cho phép người dùng định nghĩa, triển khai và quản lý hạ tầng bằng mã trên nhiều nền tảng khác nhau, từ hệ thống on-premise (máy chủ vật lý, ảo hóa) cho đến các dịch vụ điện toán đám mây như AWS, Azure, Google Cloud, hay Kubernetes.

Terraform giúp tự động hóa toàn bộ quá trình tạo lập và cấu hình tài nguyên hạ tầng, đảm bảo tính nhất quán, khả năng mở rộng, và dễ dàng kiểm soát phiên bản. Thay vì phải thao tác thủ công thông qua giao diện đồ họa hoặc lệnh CLI, người dùng chỉ cần mô tả trạng thái mong muốn của hệ thống trong các tệp cấu hình, và Terraform sẽ tự động thực hiện mọi bước cần thiết để đạt được trạng thái đó.

Terraform nổi bật với khả năng làm việc đa nền tảng thông qua hệ thống provider, cho phép quản lý tài nguyên trên nhiều môi trường khác nhau cùng lúc.

→ Ví dụ: cấu hình mạng trên AWS, triển khai container trên K3s, và tạo volume lưu trữ trên DigitalOcean.

Terraform bao gồm ba thành phần cốt lõi:

1. Configuration Files (Tập cấu hình):

Đây là nơi mô tả toàn bộ hạ tầng cần triển khai, được viết bằng ngôn ngữ **HCL (HashiCorp Configuration Language)**. Các tệp này định nghĩa các thành phần như máy chủ (instance), mạng (network), container, load balancer, volume, và các biến đầu vào.

2. State File (Tập trạng thái):

Terraform duy trì một tệp có tên **terraform.tfstate** để lưu trữ trạng thái hiện tại của hạ tầng. Nhờ đó, công cụ có thể so sánh giữa cấu hình mong muốn và trạng thái thực tế, từ đó xác định những thay đổi cần áp dụng (thêm, sửa, xóa tài nguyên).

3. Provider và Module:

- **Provider:** Là plugin kết nối Terraform với nền tảng cụ thể, ví dụ aws, google, azure, kubernetes, docker, hay k3s.
- **Module:** Là nhóm các tệp cấu hình được tái sử dụng, giúp tổ chức mã IaC rõ ràng và dễ bảo trì.

Quy trình hoạt động cơ bản của Terraform thường gồm bốn bước chính:

1. Khởi tạo (Init):

Lệnh terraform init được sử dụng để khởi tạo môi trường làm việc, tải về các provider và module cần thiết.

2. Lập kế hoạch (Plan):

Lệnh terraform plan cho phép Terraform so sánh cấu hình mô tả với trạng thái hiện tại của hạ tầng, sau đó hiển thị danh sách các thay đổi sẽ được thực hiện.

3. Triển khai (Apply):

Lệnh terraform apply thực hiện các thay đổi được xác định ở bước “plan”, bao gồm việc tạo, cập nhật hoặc xóa tài nguyên để đạt trạng thái mong muốn.

4. Hủy hạ tầng (Destroy):

Khi không còn cần thiết, người dùng có thể sử dụng terraform destroy để gỡ bỏ toàn bộ tài nguyên đã triển khai một cách an toàn và có kiểm soát.

Trong phạm vi của đề tài, **Terraform** được sử dụng như công cụ Infrastructure as Code (IaC) nhằm định nghĩa, triển khai và quản lý hạ tầng hệ thống K3s một cách tự động và nhất quán. Cụ thể, Terraform đảm nhận các nhiệm vụ chính sau:

- **Tự động khởi tạo cụm K3s** gồm các node Master và Worker, đồng thời thiết lập các thành phần cần thiết như mạng (networking), ingress controller, persistent storage và các namespace cho ứng dụng.
- **Quản lý hạ tầng dưới dạng mã**, giúp dễ dàng mở rộng, thu hẹp hoặc tái tạo lại toàn bộ môi trường triển khai khi cần thiết.
- **Đảm bảo tính đồng nhất giữa các môi trường** (phát triển, kiểm thử, triển khai thực tế) thông qua việc sử dụng cùng một bộ tệp cấu hình Terraform.
- **Tích hợp vào pipeline CI/CD**: trước khi ứng dụng được triển khai, pipeline sẽ tự động kiểm tra và áp dụng cấu hình IaC để bảo đảm hạ tầng luôn sẵn sàng, đúng phiên bản và ổn định.
- **Hỗ trợ khôi phục hạ tầng (infrastructure recovery)**: trong trường hợp hạ tầng gặp sự cố nghiêm trọng, người quản trị có thể dựa vào tệp cấu hình và trạng thái (tfstate) để tái tạo lại toàn bộ cụm K3s về trạng thái ổn định trước đó.

2.2.2 Docker

Docker là nền tảng **container hóa (containerization)** mã nguồn mở, cho phép đóng gói ứng dụng cùng toàn bộ môi trường phụ thuộc của nó (thư viện, cấu hình, biến môi trường, hệ điều hành...) vào trong một **container** độc lập.

Nhờ đó, ứng dụng có thể chạy ổn định trên nhiều môi trường khác nhau, từ máy phát triển (development) đến máy chủ triển khai (production), mà không gặp lỗi do khác biệt hệ thống.

Docker bao gồm ba thành phần chính:

- **Docker Client**: Là công cụ dòng lệnh mà người dùng tương tác để quản lý container (ví dụ: docker build, docker run, docker push).
- **Docker Daemon**: Chạy ngầm trong hệ thống, chịu trách nhiệm xử lý các yêu cầu từ client và thực hiện các thao tác trên container hoặc image.
- **Docker Hub / Registry**: Là kho lưu trữ các **Docker image**, nơi người dùng có thể tải lên (push) hoặc tải xuống (pull) các image để triển khai.

Docker Image và Container:

- **Docker Image**: Là bản mô tả tĩnh của ứng dụng, bao gồm mã nguồn, thư viện và môi trường chạy. Image thường được định nghĩa thông qua tệp **Dockerfile**.
- **Docker Container**: Là thể hiện (instance) của một Docker image đang chạy. Nhiều container có thể được khởi tạo từ cùng một image mà vẫn hoạt động độc lập.

Ưu điểm của Docker:

- **Tính di động cao:** Chạy được trên mọi hệ điều hành và nền tảng hỗ trợ Docker Engine.
- **Hiệu quả tài nguyên:** Container nhẹ hơn so với máy ảo vì chia sẻ nhân hệ điều hành (kernel).
- **Tính nhất quán:** Đảm bảo ứng dụng chạy giống nhau ở mọi môi trường.
- **Tích hợp CI/CD:** Dễ dàng đóng gói và triển khai tự động trong pipeline phát triển phần mềm.

Trong đồ án này, Docker được sử dụng để đóng gói ứng dụng thành các Docker image. Việc sử dụng Docker giúp:

- Đảm bảo tính ổn định giữa các môi trường (dev, staging, production).
- Hỗ trợ quá trình triển khai tự động thông qua CI/CD pipeline.
- Là nền tảng cho việc triển khai ứng dụng trên hệ thống K3s sau này.

2.2.3 Kubernetes light (K3s)

Kubernetes (K8s) là hệ thống mã nguồn mở do Google phát triển, dùng để tự động triển khai, mở rộng (scaling) và quản lý container trên nhiều máy chủ.

Kubernetes giúp duy trì trạng thái ổn định của hệ thống, tự động khởi động lại container khi lỗi, cân bằng tải, và hỗ trợ cập nhật phiên bản không gián đoạn.

Các thành phần chính:

- **Master Node:** Quản lý toàn bộ cụm (cluster), bao gồm API Server, Controller Manager, Scheduler và etcd.
- **Worker Node:** Chạy container thực tế thông qua Kubelet và Kube Proxy.
- **Pod:** Là đơn vị nhỏ nhất trong Kubernetes, chứa một hoặc nhiều container cùng chia sẻ tài nguyên và mạng.
- **Deployment, Service, Ingress:** Các đối tượng dùng để triển khai, cân bằng tải và điều phối ứng dụng.

K3s là phiên bản nhẹ (lightweight) của Kubernetes, được phát triển bởi Rancher Labs nhằm tối ưu cho các hệ thống có tài nguyên hạn chế (như thiết bị IoT, máy ảo nhỏ, hay môi trường học tập).

K3s được rút gọn nhưng vẫn tương thích hoàn toàn với Kubernetes chuẩn, giúp người dùng triển khai và thử nghiệm dễ dàng hơn.

Đặc điểm nổi bật của K3s:

- **Nhẹ và nhanh:** Chỉ cần một tệp nhị phân duy nhất, dung lượng nhỏ (~50MB).
- **Đơn giản trong cài đặt:** Cấu hình cụm (cluster) chỉ bằng một vài lệnh đơn giản.
- **Tích hợp sẵn các thành phần cần thiết:** như container runtime (containerd), ingress controller (Traefik), và storage backend (SQLite).
- **Tương thích hoàn toàn với Kubernetes chuẩn**, nên có thể chạy mọi ứng dụng được thiết kế cho K8s.

Mối quan hệ giữa K3s và Docker

Docker chịu trách nhiệm đóng gói ứng dụng thành image, còn K3s là hệ thống điều phối (orchestration) — triển khai, quản lý và giám sát các container đó. Khi CI/CD pipeline đẩy image mới lên Docker Registry, K3s sẽ:

- Tự động kéo image mới về.
- Tạo Pod hoặc Deployment mới dựa trên image đó.
- Giám sát tình trạng hoạt động và tự phục hồi nếu có lỗi xảy ra.

Trong đồ án này, **K3s được sử dụng để triển khai và quản lý ứng dụng** theo mô hình Kubernetes thực tế nhưng nhẹ hơn, phù hợp với môi trường mô phỏng. Cụ thể:

- Mô phỏng hệ thống gồm 1 master node và 1 worker node, đảm bảo khả năng mở rộng và quản lý tập trung.
- Cho phép triển khai tự động qua GitHub Actions khi có thay đổi trong mã nguồn.
- Hỗ trợ rollback nhanh thông qua việc đổi lại image phiên bản trước đó nếu phát hiện lỗi triển khai.

2.2.4 Github Actions

GitHub Actions là một dịch vụ tự động hóa quy trình phát triển phần mềm (CI/CD) được tích hợp trực tiếp trong nền tảng GitHub.

Công cụ này cho phép người phát triển định nghĩa và thực thi các workflow (quy trình công việc) để tự động hóa các tác vụ như:

- Kiểm tra và kiểm thử mã nguồn (testing).
- Xây dựng và đóng gói ứng dụng (build & package).
- Đóng gói image Docker và đẩy lên registry.
- Triển khai ứng dụng (deployment) lên máy chủ hoặc hệ thống Kubernetes.

Nhờ được tích hợp sẵn trong GitHub, Actions giúp người phát triển triển khai CI/CD mà không cần công cụ bên ngoài, giảm độ phức tạp và tăng khả năng cộng tác trong dự án.

Kiến trúc và thành phần chính của GitHub Actions

Một quy trình (workflow) trong GitHub Actions được định nghĩa bằng tệp YAML, thường được đặt tại đường dẫn:

```
.github/workflows/<tên_workflow>.yml
```

Mỗi workflow bao gồm các thành phần chính sau:

- **Event (Trigger):** Là sự kiện kích hoạt workflow, ví dụ:
 - + push: khi có commit mới được đẩy lên nhánh.
 - + pull_request: khi tạo hoặc cập nhật pull request.
 - + schedule: chạy tự động theo thời gian định trước.
- **Job:** Là một tập hợp các bước (steps) được thực thi trên cùng một môi trường. Mỗi job có thể chạy **độc lập hoặc song song**, tùy thuộc vào cấu hình.
- **Step:** Là các thao tác cụ thể trong job, như: chạy lệnh shell, gọi script, build Docker image, hoặc deploy ứng dụng.
- **Runner:** Là môi trường thực thi các job. GitHub cung cấp sẵn **runner ảo (hosted runner)** với nhiều hệ điều hành (Ubuntu, Windows, macOS), hoặc người dùng có thể tự cấu hình **self-hosted runner** để chạy workflow trên máy chủ cá nhân.

Một workflow CI/CD thông thường gồm các giai đoạn sau:

1. Continuous Integration (CI):

- Khi lập trình viên commit mã nguồn lên GitHub, workflow được kích hoạt tự động.

- Hệ thống sẽ thực hiện các bước như: kiểm tra cú pháp, chạy unit test, và build ứng dụng.
- Nếu tất cả kiểm tra đều thành công, workflow sẽ tạo ra **artifact** hoặc **Docker image** để phục vụ triển khai.

2. Continuous Deployment (CD):

- Sau khi image được build thành công, workflow sẽ **đăng tải (push)** image lên **Docker Hub** hoặc **private registry**.
- Tiếp đó, workflow có thể kết nối tới cụm **K3s** (thông qua SSH hoặc Kubernetes context) để **triển khai phiên bản mới của ứng dụng**.
- Nếu có lỗi trong quá trình triển khai, workflow có thể tự động kích hoạt cơ chế **rollback** về image phiên bản trước, giúp hệ thống duy trì trạng thái ổn định.

Trong đề tài, **GitHub Actions** là công cụ chính của quy trình CI/CD, đảm nhiệm toàn bộ quá trình tự động hóa:

1. Tự động build và kiểm thử ứng dụng sau mỗi lần cập nhật mã nguồn.
2. Đóng gói ứng dụng thành Docker image và đẩy lên Docker Hub.
3. Tự động triển khai lên cụm K3s thông qua self-hosted runner.
4. Theo dõi và rollback tự động khi phát hiện lỗi trong quá trình triển khai.

Nhờ tích hợp chặt chẽ với Docker và K3s, GitHub Actions giúp toàn bộ quy trình **triển khai trở nên tự động, đồng nhất, và đáng tin cậy**, giảm thiểu can thiệp thủ công và tăng hiệu quả phát triển phần mềm.

3. Kịch bản thử nghiệm

3.1 Deploy K3s Cluster thông qua Terraform

3.1.1 Mục tiêu

Mục tiêu của việc triển khai K3s Cluster thông qua Terraform là thiết lập một nền tảng hạ tầng Kubernetes nhất quán, có khả năng tái lập và tự động hóa hoàn toàn. Điều này nhằm cung cấp một môi trường runtime tin cậy và ổn định cho việc triển khai ứng dụng, đồng thời phục vụ như một bước chuẩn bị hạ tầng thiết yếu cho toàn bộ quy trình CI/CD được xây dựng trong dự án.

3.1.2 Thực hiện

Cấu trúc thư mục dùng để chạy Terraform:

```
myapp/
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   └── outputs.tf
```

Thực hiện chỉnh sửa cho từng file:

Với file **main.tf**:

```
resource "ssh_resource" "install_master" {
  host      = var.master_ip
  user      = var.user
  private_key = file(var.ssh_private_key)
  timeout   = "15m"
}
```

Đoạn này khai báo một resource tên là **install_master** thuộc loại **ssh_resource**. Terraform sẽ dùng nó để thực hiện các thao tác SSH lên máy chủ từ xa.

```
commands = [
...
  "curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC='server' sh -",
...
]
```

Đoạn code này dùng để install k3s master. Sau khi cài đặt thành công, terraform sẽ thực hiện

```
resource "ssh_resource" "get_token" {
  host      = var.master_ip
  user      = var.user
  private_key = file(var.ssh_private_key)
  timeout   = "5m"
}

commands = [
  "sleep 5",
  "if ! sudo test -f /var/lib/rancher/k3s/server/node-token; then echo 'ERROR: Token file not found!'; exit 1; fi",
  "sudo cat /var/lib/rancher/k3s/server/node-token"
]

depends_on = [ssh_resource.install_master]
}
```


Đoạn cấu hình này được sử dụng để **kết nối SSH vào node Master** của hệ thống K3s và **lấy ra token** cần thiết cho quá trình thêm các node Worker vào cluster. Token này nằm trong file `/var/lib/rancher/k3s/server/node-token`, được K3s tự động tạo ra sau khi Master khởi động thành công.

```
resource "ssh_resource" "install_worker" {
  host      = var.worker_ip
  user      = var.user
  private_key = file(var.ssh_private_key)
  timeout   = "15m"

  commands = [
    ...
    "curl -sL https://get.k3s.io | K3S_URL='https://${var.master_ip}:6443'
    K3S_TOKEN='${trimspace(ssh_resource.get_token.result)}' sh -",
    ...
  ]
}
```

Sau khi đã có token từ Master thì terraform sẽ tiếp tục cài đặt K3s trên máy Worker

Trên file **variables.tf**:

```
variable "master_ip" {
  description = "Địa chỉ IP của master node"
  default     = "192.168.2.61"
}

variable "worker_ip" {
  description = "Địa chỉ IP của worker node"
  default     = "192.168.2.166"
}

variable "user" {
  description = "Tên user SSH để đăng nhập vào các node"
  default     = "mnhat"
}

variable "ssh_private_key" {
  description = "Đường dẫn tới private key SSH"
  default     = "~/.ssh/id_rsa"
}

variable "cluster_name" {
  description = "Tên cụm cluster K3s"
  default     = "lan-cluster"
}
```

File **variables.tf** được sử dụng để khai báo các biến cấu hình cần thiết cho quá trình triển khai tự động bằng Terraform.

Việc sử dụng biến giúp cho mã Terraform trở nên linh hoạt, dễ bảo trì, và dễ tái sử dụng cho nhiều môi trường khác nhau (ví dụ: thay đổi IP, user, hoặc tên cluster mà không cần chỉnh sửa trực tiếp trong các file resource).

Trên file **output.tf**:

```
output "cluster_name" {
  description = "Tên cụm cluster"
  value      = var.cluster_name
}

output "master_ip" {
  description = "Địa chỉ IP của master node"
  value      = var.master_ip
}

output "worker_ip" {
  description = "Địa chỉ IP của worker node"
  value      = var.worker_ip
}

output "join_token" {
  description = "Token dùng để join worker node vào master"
  value      = trimspace(ssh_resource.get_token.result)
  sensitive  = true
}

output "k3s_url" {
  description = "Địa chỉ API của K3s Master"
  value      = "https://${var.master_ip}:6443"
}

output "kubeconfig_command" {
  description = "Lệnh để lấy kubeconfig từ master"
  value      = "ssh ${var.user}@${var.master_ip} 'sudo cat /etc/rancher/k3s/k3s.yaml'"
}
```

File **outputs.tf** được sử dụng để định nghĩa các giá trị đầu ra (output) mà Terraform sẽ hiển thị sau khi hoàn tất quá trình triển khai.

Các output này giúp người quản trị hoặc các script CI/CD khác dễ dàng truy cập thông tin cần thiết (chẳng hạn như IP của các node, token để join cluster, hay lệnh lấy kubeconfig từ node Master.)

Cách chạy terraform:

```
terraform init
terraform apply -auto-approve
```

3.1.3 Kết quả

```
mnhat@ubuntu:~/myapp/terraform$ terraform apply -auto-approve
ssh_resource.install_master: Refreshing state... [id=5577006791947779410]
ssh_resource.get_token: Refreshing state... [id=8674665223082153551]
ssh_resource.install_worker: Refreshing state... [id=5577006791947779410]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration
and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

cluster_name = "lan-cluster"
join_token = <sensitive>
k3s_url = "https://192.168.2.61:6443"
kubeconfig_command = "ssh mnhat@192.168.2.61 'sudo cat /etc/rancher/k3s/k3s.
yaml'"
master_ip = "192.168.2.61"
worker_ip = "192.168.2.166"
```

Sau khi thực hiện áp dụng các cấu hình, lúc này terraform đã tạo cho ta 1 cụm k3s gồm 2 node, đó là:

- Master node: 192.168.2.61
- Worker node: 192.168.2.166

Thực hiện kiểm tra trên máy Master node:

```
mnhat@ubuntu1:~$ sudo kubectl get nodes
NAME        STATUS    ROLES                  AGE     VERSION
ubuntu1     Ready     control-plane,master   7d9h    v1.33.5+k3s1
ubuntu2     Ready     <none>                 7d9h    v1.33.5+k3s1
```

Bảng tổng kết

Thành phần	Mô tả	Địa chỉ IP	Trạng thái	Ghi chú
Terraform	Công cụ dùng để tự động hóa việc tạo cụm k3s	–	Hoàn thành	Đã chạy terraform init, plan, apply thành công
K3s Master Node	Node chính quản lý cụm k3s	192.168.2.61	Hoàn thành	Đã cài đặt và khởi tạo cụm thành công

K3s Worker Node	Node phụ tham gia cụm k3s	192.168.2.166	Hoàn thành	Đã join vào cụm qua token từ master
Kiểm tra cụm (kubectl get nodes)	Hiển thị danh sách node trong cluster	–	Hoàn thành	Hiển thị đủ 2 node: master và worker
Kết luận	Terraform triển khai cụm k3s hoàn chỉnh gồm 2 node hoạt động ổn định	–	Hoàn thành	Sẵn sàng cho triển khai ứng dụng

3.2 Tự động hóa triển khai khi thay đổi mã nguồn

3.2.1 Mục tiêu:

Mục tiêu của nội dung này là xây dựng một quy trình triển khai tự động nhằm đảm bảo rằng mỗi khi mã nguồn trên nhánh chính (main branch) được cập nhật, hệ thống sẽ tự động thực hiện việc kiểm thử, đóng gói và triển khai ứng dụng lên môi trường chạy thực tế. Việc tự động hóa này giúp giảm thiểu các thao tác thủ công trong quá trình triển khai, đảm bảo tính nhất quán giữa các phiên bản, đồng thời tận dụng hạ tầng được định nghĩa bằng mã (Infrastructure as Code – IaC) để duy trì sự ổn định và dễ dàng mở rộng của hệ thống.

Bên cạnh đó, quá trình tích hợp liên tục (Continuous Integration) và triển khai liên tục (Continuous Deployment) giúp rút ngắn chu kỳ phát triển, tăng độ tin cậy và cho phép nhóm phát triển nhanh chóng phản hồi các thay đổi của người dùng.

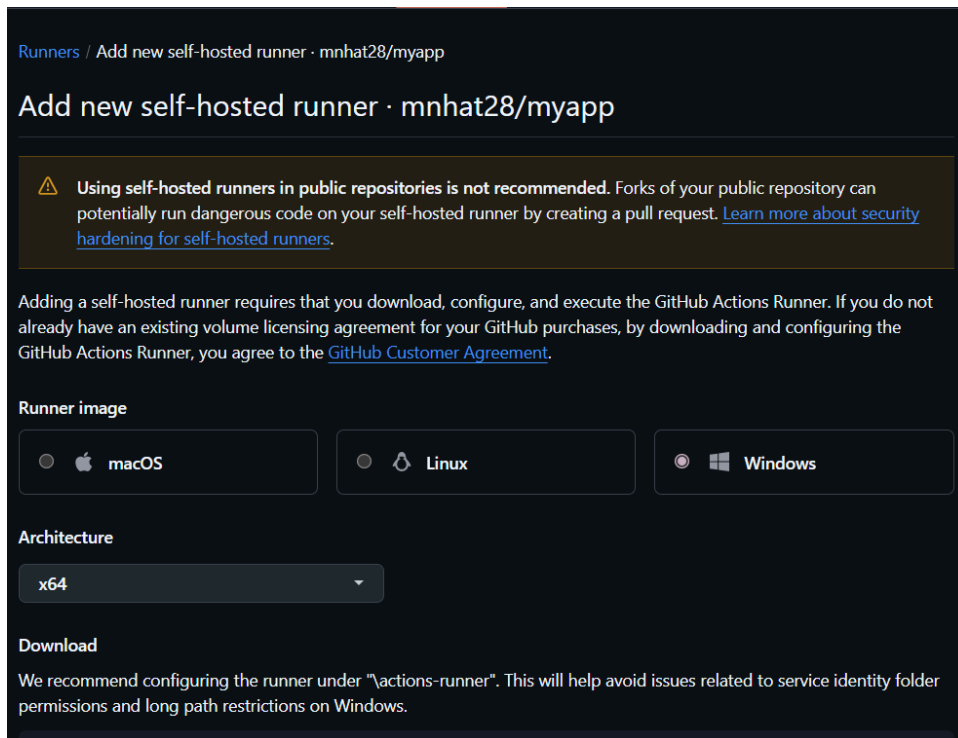
3.2.2 Thực hiện:

3.2.2.1 Cài đặt Self-hosted runner

Self-hosted runner được sử dụng để thực thi các job trong CI/CD pipeline trên chính hạ tầng máy chủ thuộc quyền sở hữu và quản lý, thay vì sử dụng hạ tầng đám mây dùng chung của GitHub.

Trong dự án này, self-hosted runner đóng vai trò then chốt trong việc triển khai ứng dụng lên cluster K3s cục bộ, cho phép truy cập trực tiếp đến môi trường mạng nội bộ, thực thi các lệnh Terraform để quản lý hạ tầng, và chạy các lệnh kubectl để triển khai ứng dụng trực tiếp lên cluster. Ngoài ra, việc sử dụng self-hosted runner còn mang lại khả năng tùy biến cao về cấu hình phần cứng, cài đặt phần mềm cần thiết, đồng thời đảm bảo tính bảo mật và kiểm soát toàn diện đối với môi trường thực thi pipeline.

Trên Github, vào repository của dự án, chọn Settings → Actions → Runners → New Self-hosted runner.



Runners / Add new self-hosted runner · mnhat28/myapp

Add new self-hosted runner · mnhat28/myapp

Warning: Using self-hosted runners in public repositories is not recommended. Forks of your public repository can potentially run dangerous code on your self-hosted runner by creating a pull request. [Learn more about security hardening for self-hosted runners.](#)

Adding a self-hosted runner requires that you download, configure, and execute the GitHub Actions Runner. If you do not already have an existing volume licensing agreement for your GitHub purchases, by downloading and configuring the GitHub Actions Runner, you agree to the [GitHub Customer Agreement](#).

Runner image

☐ macOS ☒ Linux ☐ Windows

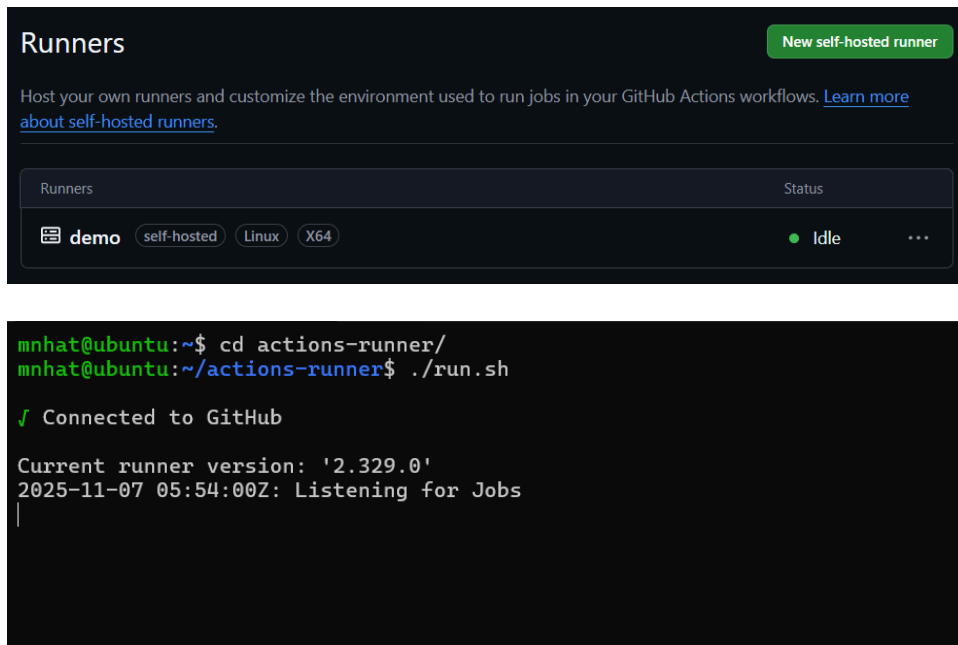
Architecture

x64

Download

We recommend configuring the runner under "actions-runner". This will help avoid issues related to service identity folder permissions and long path restrictions on Windows.

Thực hiện chọn và chạy các lệnh mà Github đưa.



Runners

Host your own runners and customize the environment used to run jobs in your GitHub Actions workflows. [Learn more about self-hosted runners.](#)

[New self-hosted runner](#)

Runners	Status
demo self-hosted Linux X64	● Idle ...

```
mnhat@ubuntu:~$ cd actions-runner/
mnhat@ubuntu:~/actions-runner$ ./run.sh

✓ Connected to GitHub

Current runner version: '2.329.0'
2025-11-07 05:54:00Z: Listening for Jobs
```

Sau khi cài đặt xong. Để sử dụng, trong máy vừa cài đặt thực hiện truy cập vào thư mục actions-runner và chạy lệnh:

```
./run.sh
```

3.2.2.2 CI/CD workflows

Trong repository, chọn Actions → New workflow → set up a workflow yourself (hoặc cũng có thể sử dụng các template có sẵn mà Github đưa)

Main.yml (Đây là file dùng để chạy CI/CD)

```
name: CI/CD to K3s Master
on:
  push:
    branches: [ "main" ] #Lưu ý: workflows sẽ được chạy mỗi khi có code mới được
    đẩy lên
jobs:
  build-and-deploy:
    runs-on: self-hosted
    steps:
      - name: Checkout code
        uses: actions/checkout@v3
      - name: Log in to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }
      - name: Build and push Docker image
        run: |
          docker build -t ${ secrets.DOCKERHUB_USERNAME }/myapp:${{
github.sha }} .
          docker push ${ secrets.DOCKERHUB_USERNAME }/myapp:${{
github.sha }}
      - name: Deploy to K3s Master via SSH
        uses: appleboy/ssh-action@v1.0.0
        with:
          host: ${ secrets.MASTER_HOST }
          username: ${ secrets.MASTER_USER }
          key: ${ secrets.MASTER_SSH_KEY }
          script: |
            cd ~/k8s
            sed -i "s|image:.*myapp:.*|image: ${ secrets.DOCKERHUB_USERNAME
}}/myapp:${{ github.sha }}|g" deployment.yaml
            sudo kubectl apply -f deployment.yaml
            sudo kubectl apply -f service.yaml
            sudo kubectl apply -f ingress.yaml
            sudo kubectl annotate deployment myapp
            kubernetes.io/change-cause="Deploy image: ${
secrets.DOCKERHUB_USERNAME }/myapp:${{ github.sha }}" --overwrite
```

```

echo "Waiting for rollout to complete..."
if ! sudo kubectl rollout status deployment/myapp -n default --timeout=180s;
then
    echo "Rollout failed! Performing rollback..."

    ...

    sudo kubectl rollout undo deployment/myapp -n default
    sudo kubectl rollout status deployment/myapp -n default --timeout=120s
    echo "Rollback completed."
    exit 1
fi

echo "Deployment successful."

```

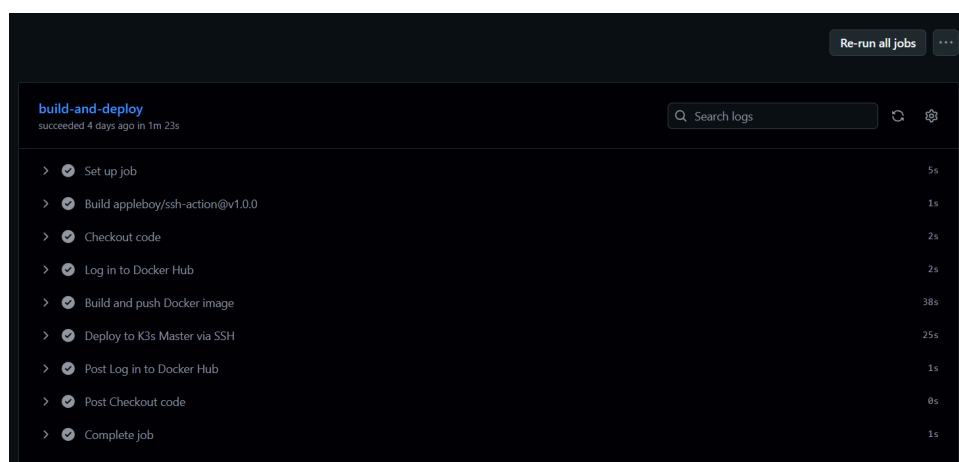
Sau khi thực hiện viết xong kịch bản, ta commit code bên main branch.

Lưu ý: branches: ["main"] dòng này trong đoạn code trên sẽ giúp cho workflows tự động chạy mỗi khi có code mới được push lên Github. Đây là một thao tác quan trọng giúp cho người vận hành không cần phải sau khi push code lên Github rồi phải chạy workflows thủ công nữa.

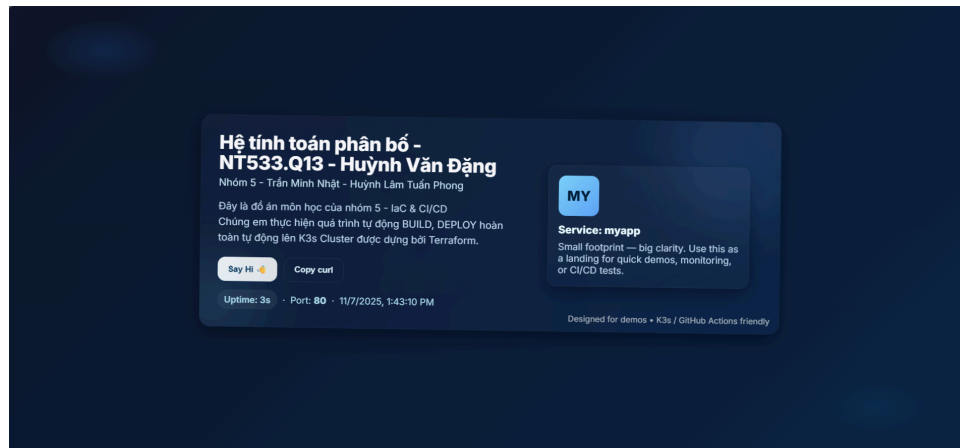
Về kịch bản của workflow:

- Đầu tiên, máy runner sẽ dùng những thông tin được lưu trong Secrets (đây là nơi lưu trữ các thông tin nhạy cảm, nếu đưa các thông tin nhạy cảm vào thẳng đoạn code sẽ dễ bị lộ thông) để đăng nhập vào Dockerhub.
- Tiếp đến, nó sẽ thực hiện build docker image thông qua dockerfile rồi push lên Dockerhub và gắn tag là github.sha (là mã băm của commit hiện tại).
- Sau cùng, máy runner sẽ thực hiện SSH vào máy master để sửa lại những thông tin của image rồi chạy các file deployment, service, ingress để triển khai trang web.

3.2.3 Kết quả



Ta có thể giám sát quá trình CI/CD để có thể biết được workflow bị lỗi ở vị trí nào. Sau khi workflow chạy thành công, ta quan sát được lúc này web đã được triển khai lên local.



Về sau, mỗi khi chỉnh sửa 1 file nào chỉnh cần commit lên lại Github, workflow sẽ tự động chạy mà không cần phải triển khai thủ công nữa.

Bảng tổng kết

Thành phần	Mô tả	Trạng thái thực thi	Ghi chú
Self-hosted Runner	Runner nội bộ được cài đặt trên máy chủ cục bộ để thực thi pipeline và truy cập trực tiếp cụm K3s.	Hoàn thành	Đã khởi chạy thành công với lệnh ./run.sh.
Quy trình CI/CD (Build → Push → Deploy)	Khi có commit mới lên GitHub, workflow tự động đăng nhập Docker Hub, build image từ Dockerfile, push image lên Docker Hub và triển khai lên cụm K3s thông qua SSH vào node master.	Hoàn thành	Ứng dụng được triển khai thành công lên cụm K3s với IP master 192.168.2.61 và worker 192.168.2.166.
Kiểm tra trạng thái Workflow	Theo dõi tiến trình thực thi và log của pipeline trên GitHub Actions.	Hoàn thành	Workflow tự động kích hoạt sau mỗi commit mới.

Kết luận	Hệ thống CI/CD hoạt động hoàn chỉnh, đảm bảo quy trình build – push – deploy tự động và ổn định.	Hoàn thành	Đạt mục tiêu tự động hoá triển khai ứng dụng lên cụm K3s.
-----------------	--	------------	---

3.3 Rollback khi production bị lỗi hoặc khi deploy lỗi

3.3.1 Mục tiêu

Trong quy trình CI/CD, việc đảm bảo tính ổn định của môi trường production là yếu tố cốt lõi. Khi triển khai phiên bản mới, rủi ro lỗi xảy ra (do cấu hình sai, lỗi build, lỗi ứng dụng, v.v.) có thể làm gián đoạn dịch vụ.

Mục tiêu của phần này là thiết kế cơ chế triển khai tự động kết hợp rollback tự động trên cụm K3s, cho phép hệ thống:

- Tự động cập nhật phiên bản Docker image mới khi có thay đổi mã nguồn.
- Theo dõi quá trình triển khai (rollout) và xác định lỗi.
- Tự động khôi phục (rollback) về phiên bản image ổn định trước đó nếu phát hiện lỗi.

Cơ chế này giúp đảm bảo tính sẵn sàng (availability) và độ tin cậy (reliability) của hệ thống trong suốt vòng đời triển khai.

3.3.2 Thực hiện

3.3.2.1 Theo dõi rollout và xử lý lỗi triển khai

Sau khi triển khai image mới, pipeline sẽ theo dõi tiến trình rollout của deployment:

```
sudo kubectl rollout status deployment/myapp -n default --timeout=180s
```

Lệnh này giúp xác minh xem các Pod mới có được khởi tạo thành công hay không.

Nếu trong vòng 180 giây rollout không hoàn tất (ví dụ Pod bị crash hoặc image lỗi), pipeline sẽ tự động kích hoạt cơ chế rollback.

3.3.2.2 Rollback tự động khi triển khai thất bại

Khi phát hiện lỗi, pipeline sẽ:

1. Ghi chú nguyên nhân rollback vào annotation:

```
TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')
sudo kubectl annotate deployment myapp \
kubernetes.io/change-cause="Rollback on $TIMESTAMP due to failed rollout of
image: ${ secrets.DOCKERHUB_USERNAME }}/myapp:${ github.sha }" \
--overwrite
```

2. Thực hiện rollback về phiên bản image ổn định trước đó:

```
sudo kubectl rollout undo deployment/myapp -n default
```

3. Theo dõi tiến trình khôi phục:

```
sudo kubectl rollout status deployment/myapp -n default --timeout=120s
```

Nếu rollback hoàn tất, hệ thống in ra thông báo **"Rollback completed."** và dừng pipeline bằng exit 1, nhằm đánh dấu rằng phiên bản mới triển khai thất bại, nhưng môi trường production đã được phục hồi an toàn.

3.3.3 Kết quả

```
68 out: deployment.apps/myapp configured
69 out: service/myapp-service unchanged
70 out: ingress.networking.k8s.io/myapp-ingress unchanged
71 out: deployment.apps/myapp annotated
72 out: 🟡 Waiting for rollout to complete...
73 out: Waiting for deployment "myapp" rollout to finish: 1 out of 3 new replicas have been updated...
74 out: Waiting for deployment "myapp" rollout to finish: 1 out of 3 new replicas have been updated...
75 out: Waiting for deployment "myapp" rollout to finish: 2 out of 3 new replicas have been updated...
76 out: Waiting for deployment "myapp" rollout to finish: 2 out of 3 new replicas have been updated...
77 out: Waiting for deployment "myapp" rollout to finish: 2 out of 3 new replicas have been updated...
78 err: error: timed out waiting for the condition
79 out: ❌ Rollout failed! Performing rollback...
80 out: deployment.apps/myapp annotated
81 out: deployment.apps/myapp rolled back
82 out: Waiting for deployment "myapp" rollout to finish: 1 old replicas are pending termination...
83 out: Waiting for deployment "myapp" rollout to finish: 1 old replicas are pending termination...
84 out: Waiting for deployment "myapp" rollout to finish: 1 old replicas are pending termination...
85 out: deployment "myapp" successfully rolled out
86 out: ✅ Rollback completed.
87 2025/11/04 06:28:32 Process exited with status 1
```

Kết quả thử nghiệm cho thấy pipeline CI/CD có khả năng tự động phát hiện và khắc phục lỗi triển khai một cách hiệu quả.

Khi rollout của phiên bản mới thất bại, pipeline ngay lập tức khôi phục lại Docker image của phiên bản trước, giúp dịch vụ duy trì hoạt động liên tục mà không xảy ra downtime đáng kể.

Cơ chế ghi chú và lưu lịch sử triển khai giúp nhóm phát triển dễ dàng truy xuất và kiểm tra lại các phiên bản trước thông qua lệnh:

```
kubectl rollout history deployment/myapp
```

Nhờ đó, hệ thống triển khai trên K3s không chỉ đảm bảo tự động hóa hoàn toàn quy trình phát hành, mà còn duy trì được tính ổn định, khả năng phục hồi và an toàn cao trong môi trường production.

Bảng tổng kết

Thành phần	Mô tả	Trạng thái thực thi	Ghi chú
Theo dõi Rollout	Pipeline theo dõi quá trình triển khai ứng dụng bằng lệnh kubectl rollout status để phát hiện sớm lỗi khi khởi tạo Pod.	Hoàn thành	Phát hiện chính xác các lỗi rollout trong thời gian chờ (timeout 180s).
Kích hoạt Rollback tự động	Khi rollout thất bại, pipeline tự động kích hoạt rollback về phiên bản image ổn định trước đó.	Hoàn thành	Cơ chế rollback được thực thi tự động bằng kubectl rollout undo.
Ghi chú lịch sử triển khai	Ghi lại nguyên nhân và thời gian rollback bằng annotation kubernetes.io/change-cause.	Hoàn thành	Hỗ trợ truy xuất lịch sử triển khai qua kubectl rollout history.
Khôi phục dịch vụ Production	Sau rollback, dịch vụ được phục hồi an toàn, không xảy ra downtime đáng kể.	Hoàn thành	Đảm bảo tính availability và reliability của hệ thống.
Kết luận	Hệ thống CI/CD có khả năng tự phát hiện lỗi, tự rollback và duy trì ổn định môi trường production.	Hoàn thành	Đạt mục tiêu tự động hóa và phục hồi an toàn khi triển khai lỗi.

3.4 High Availability & Load Balancer bằng thuật toán chia tải Round Robin

3.4.1 Mục tiêu

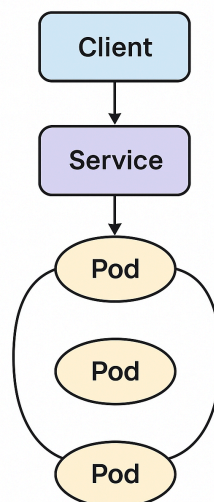
Khi ứng dụng được triển khai trong môi trường container trên cụm K3s, việc đảm bảo khả năng phục vụ đồng thời cho nhiều người dùng là yếu tố then chốt để nâng cao hiệu năng và độ sẵn sàng (High Availability) của hệ thống.

Mục tiêu của phần này là thiết lập **cơ chế cân bằng tải (Load Balancer)** sử dụng **thuật toán Round Robin** nhằm phân phối đồng đều lượng truy cập giữa các Pod trong cluster. Thông qua đó, hệ thống đạt được các mục tiêu cụ thể sau:

- **Phân phối tải đồng đều:** Mỗi Pod trong cụm sẽ xử lý một phần công bằng trong tổng số yêu cầu (request) của người dùng, tránh tình trạng dồn tải lên một Pod duy nhất.
- **Giảm thiểu tình trạng quá tải cục bộ (Bottleneck):** Đảm bảo các tài nguyên được sử dụng hiệu quả, giúp hệ thống duy trì hiệu năng ổn định ngay cả khi số lượng truy cập tăng cao.
- **Tăng khả năng mở rộng (Scalability):** Khi số lượng bản sao (replica) tăng, cơ chế Load Balancer tự động điều phối lưu lượng đến các Pod mới mà không cần can thiệp thủ công.
- **Đảm bảo tính sẵn sàng cao (High Availability):** Khi một node hoặc Pod gặp sự cố, các Pod còn lại vẫn tiếp tục phục vụ yêu cầu, giúp hệ thống hoạt động liên tục mà không bị gián đoạn.

Nhờ cơ chế này, hệ thống triển khai trên K3s có thể hoạt động ổn định, hiệu suất cao và sẵn sàng đáp ứng nhu cầu truy cập lớn trong môi trường thực tế.

Load Balancer
(Round Robin)



3.4.2 Thực hiện

3.4.2.1 Traefik

Giới thiệu về Traefik

Traefik là một reverse proxy và load balancer hiện đại, được thiết kế để hoạt động tối ưu trong các môi trường container và microservices như Docker, Kubernetes hay k3s. Traefik có khả năng tự động phát hiện và định tuyến lưu lượng truy cập đến đúng dịch vụ mà không cần cấu hình thủ công. Bên cạnh đó, nó còn hỗ trợ cấp chứng chỉ SSL tự động, cấu hình động, và cung cấp dashboard trực quan giúp giám sát luồng yêu cầu. Trong hệ thống k3s, Traefik thường được cài đặt sẵn và đảm nhiệm vai trò Ingress Controller, cho phép quản lý việc truy cập vào các ứng dụng trong cluster một cách linh hoạt và an toàn.

Traefik trong High Availability (HA)

Trong mô hình High Availability, Traefik có thể triển khai theo dạng cụm nhiều node nhằm đảm bảo tính sẵn sàng cao cho hệ thống. Các instance của Traefik có thể chia sẻ trạng thái phiên hoạt động thông qua các backend lưu trữ như Consul, etcd, hoặc Redis. Khi một node Traefik gặp sự cố, các node còn lại sẽ tự động tiếp quản, đảm bảo luồng truy cập của người dùng không bị gián đoạn. Nhờ đó, hệ thống luôn duy trì hoạt động ổn định, giảm thiểu tối đa thời gian downtime.

Traefik trong Load Balancing

Traefik tích hợp sẵn cơ chế cân bằng tải (Load Balancing) giúp phân phối lưu lượng truy cập giữa nhiều instance của cùng một dịch vụ. Nó hỗ trợ nhiều thuật toán cân bằng tải như Round Robin, Weighted Round Robin và Sticky Sessions. Ngoài ra, Traefik có khả năng kiểm tra tình trạng (health check) của các dịch vụ để loại bỏ tạm thời những instance gặp lỗi, đảm bảo lưu lượng chỉ được gửi đến các node hoạt động tốt. Nhờ khả năng này, Traefik giúp tối ưu hiệu năng, tăng tính ổn định và khả năng mở rộng cho toàn bộ hệ thống.

3.4.2.2 Cách thực hiện

Trước hết, thực hiện thêm 1 máy mới vào cụm để đảm bảo cụm có 3 máy hoạt động bao gồm 1 master và 2 worker.

Trong máy master, thực hiện chỉnh sửa file service.yaml với các nội dung sau:

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  selector:
    app: myapp
  type: ClusterIP
```

```
ports:
  - port: 80
    targetPort: 80
```

Chỉnh sửa type từ NodePort thành ClusterIP để tạo một địa chỉ IP nội bộ cố định cho Service, giúp các ứng dụng khác bên trong cluster Kubernetes có thể khám phá và giao tiếp với tập hợp các Pod được chọn bởi Service này.

Thực hiện tạo file service.yaml với nội dung:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
  annotations:
    kubernetes.io/ingress.class: "traefik"
spec:
  rules:
  - host: myapp.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: myapp-service # phải trùng với tên service.yaml
            port:
              number: 80
```

Lưu ý: Lúc này, ta có thể truy cập trực tiếp vào cụm bằng tên miền là myapp.local. Đây là một tên miền ảo do em tạo, nên muốn truy cập vào tên miền này, trên các máy client phải thêm host myapp.local với địa chỉ IP là IP của các máy trong cụm K3s.

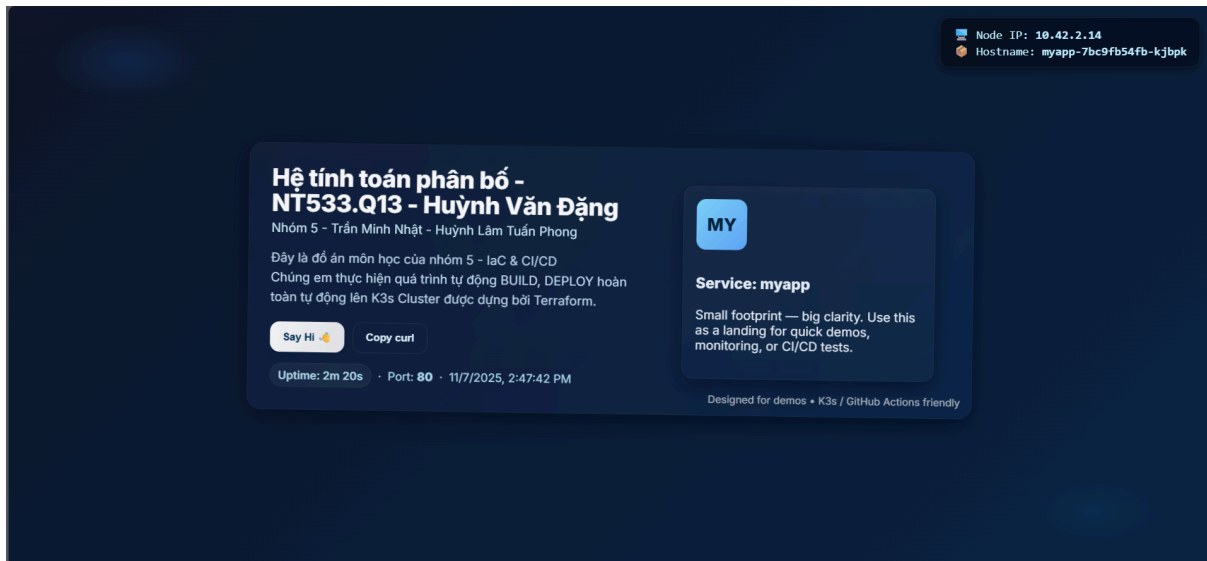
3.4.3 Kết quả

3.4.3.1 Load Balance

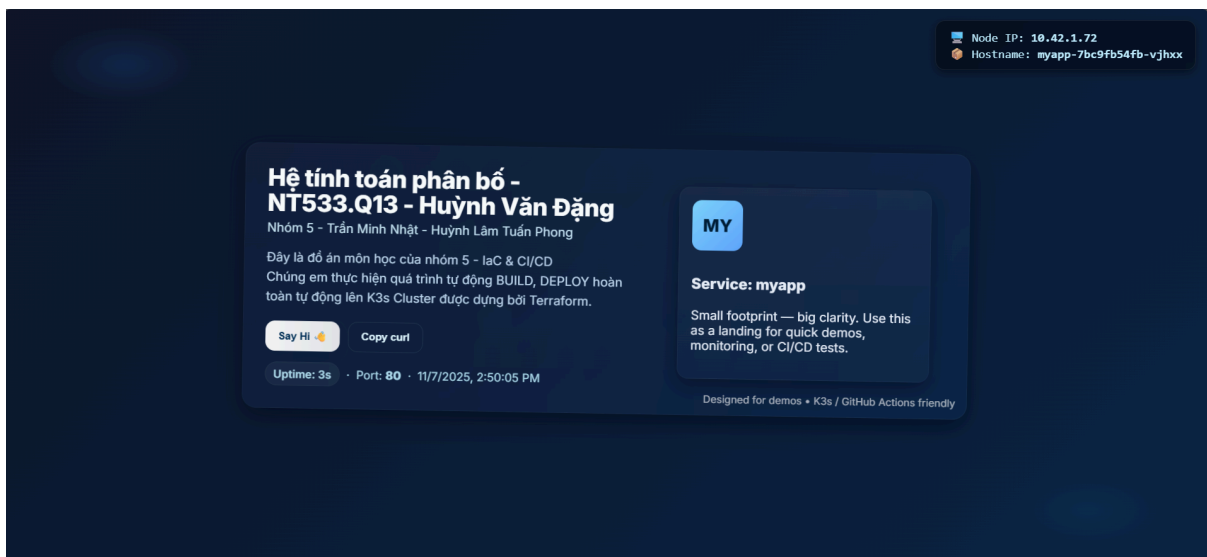
```
mhath@ubuntu1:~$ sudo kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS
GATES								
myapp-7bc9fb54fb-grfpj	1/1	Running	0	113s	10.42.0.115	ubuntu1	<none>	<none>
myapp-7bc9fb54fb-kjbpk	1/1	Running	0	2m15s	10.42.2.14	ubuntu	<none>	<none>
myapp-7bc9fb54fb-vjhxx	1/1	Running	0	2m2s	10.42.1.72	ubuntu2	<none>	<none>

Thực hiện kiểm thử trên máy master, lúc này ta thấy các pods đã được hoạt động.



Quan sát ở góc trên màn hình, ta sẽ thấy được địa chỉ IP của các node. Hình ảnh trên cho thấy, web này đang được triển khai bởi pod có IP là 10.42.2.14. Và mỗi lần reload lại trang web địa chỉ IP sẽ thay đổi mới.

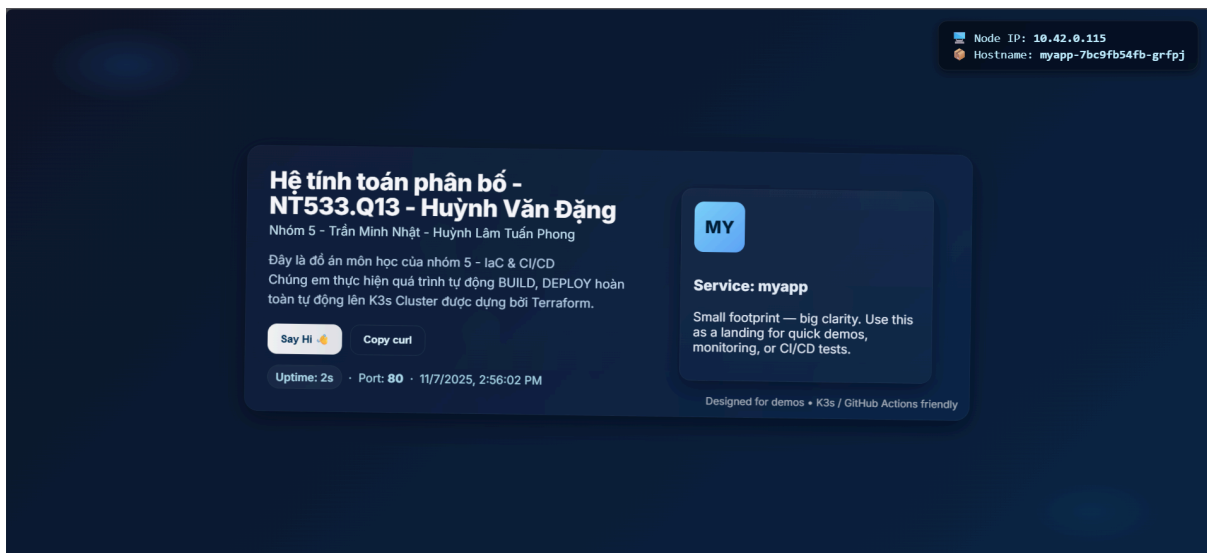
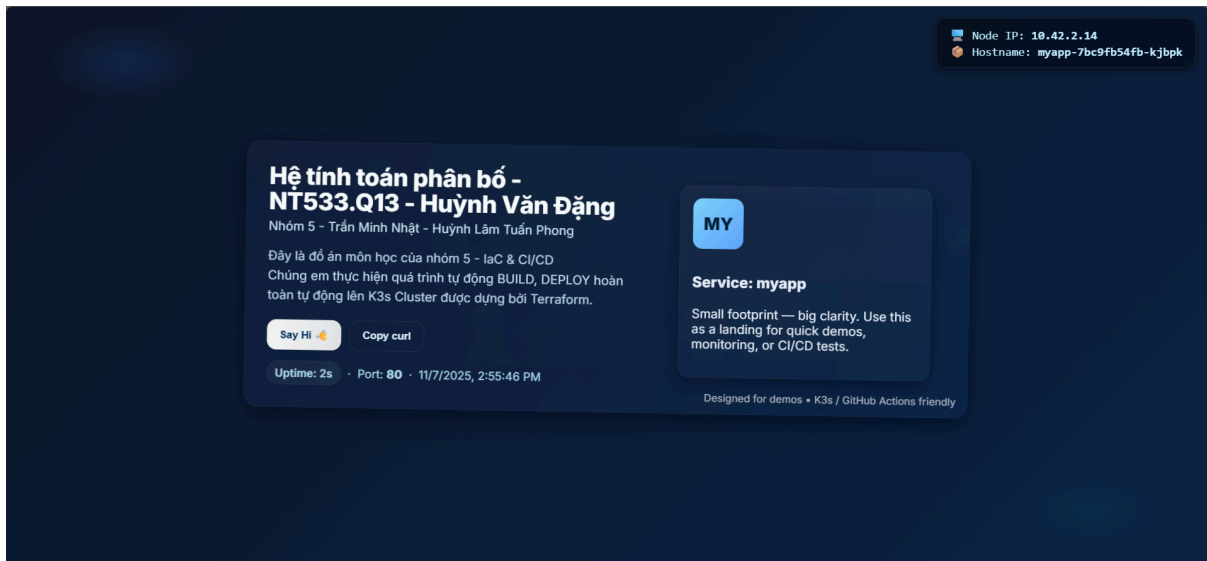


Điều này cho thấy được, luồng dữ liệu truy cập vào cụm sẽ được điều hướng đều đến các pods thông qua thuật toán Round Robin. Từ đó có giúp cho các pod luôn đạt hiệu suất ổn định, tránh bị quá tải.

3.4.3.2 High Availability

```
mnhat@ubuntu1:~$ sudo kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
ubuntu     Ready     <none>   3d2h  v1.33.5+k3s1
ubuntu1    Ready     control-plane,master  27d   v1.33.5+k3s1
ubuntu2    NotReady  <none>   27d   v1.33.5+k3s1
```

Lúc này, khi thực hiện tắt đi 1 nodes thì trang web vẫn hoạt động bình thường.



Khi reload lại trang liên tục, ta chỉ còn thấy 2 địa chỉ IP là 10.42.0.115 và 10.42.2.14. Từ đây cho thấy, các luồng traffic sẽ được điều hướng đến những node, pod còn hoạt động. Nhờ vậy, mà khi có 1 node, pod không hoạt động thì hệ thống vẫn hoạt động bình thường.

Bảng tổng kết

Thành phần	Mô tả	Kết quả quan sát	Trạng thái
Triển khai Traefik	Traefik được sử dụng làm Ingress Controller kiêm Load Balancer, giúp định tuyến và phân phối lưu lượng đến các Pod trong cụm K3s.	Hệ thống định tuyến ổn định, tự động nhận diện các dịch vụ.	Hoàn thành

Cấu hình Service (ClusterIP)	Service được chỉnh type thành ClusterIP để cho phép các Pod nội bộ giao tiếp qua IP cố định.	Các Pod trong cluster giao tiếp thành công thông qua IP nội bộ.	Hoàn thành
Cấu hình Ingress (Traefik)	Cấu hình Ingress với tên miền nội bộ myapp.local, sử dụng annotation kubernetes.io/ingress.class: "traefik".	Truy cập web thành công qua myapp.local từ client nội bộ.	Hoàn thành
Kiểm thử Load Balancing (Round Robin)	Reload trang web nhiều lần cho thấy IP Pod thay đổi luân phiên.	Lưu lượng được phân phối đều giữa các Pod (ví dụ: 10.42.2.14 → 10.42.0.115 → ...).	Hoàn thành
Kiểm thử High Availability (HA)	Tắt một node trong cụm để kiểm tra khả năng phục hồi.	Ứng dụng vẫn truy cập bình thường, Traefik tự động điều hướng sang node còn hoạt động.	Hoàn thành
Kết luận	Cụm K3s đạt được tính cân bằng tải (Load Balancing) và tính sẵn sàng cao (High Availability) thông qua Traefik và thuật toán Round Robin.	Hệ thống hoạt động ổn định, hiệu suất tốt.	Hoàn thành

3.5 Video thực hiện

Video thực hiện: [Demo](#)

4. Tổng kết

Nội dung thực hiện	Công cụ / Giải pháp sử dụng	Kết quả đạt được	Ghi chú
Xây dựng hạ tầng K3s Cluster	Terraform	Tự động tạo và cấu hình cụm K3s gồm 1 Master và 2 Worker nodes.	Cụm hoạt động ổn định, có thể mở rộng linh hoạt.
Triển khai CI/CD Pipeline	GitHub Actions (Self-hosted Runner)	Tự động build Docker image, push lên Docker Hub và triển khai lên K3s khi có commit mới.	Loại bỏ hoàn toàn thao tác thủ công, tăng hiệu quả DevOps.
Triển khai ứng dụng container	Docker + K3s	Ứng dụng được container hóa và triển khai thành công trên cụm K3s.	Ứng dụng chạy ổn định, có thể truy cập qua Ingress.
Cơ chế Rollback tự động	GitHub Actions + kubectl rollout	Khi phát hiện lỗi triển khai, hệ thống tự động phục hồi phiên bản trước.	Đảm bảo tính availability và reliability của production.
Load Balancing (Round Robin)	Traefik Ingress Controller	Phân phối đều lưu lượng truy cập giữa các Pod.	Giúp tăng hiệu suất và tránh quá tải cục bộ.
High Availability (HA)	Traefik Cluster	Hệ thống vẫn hoạt động khi một node gặp sự cố.	Giảm downtime, tăng độ sẵn sàng.
Giám sát và kiểm thử	Dashboard Traefik, kubectl logs/status	Theo dõi và xác minh quá trình triển khai, rollback, cân bằng tải.	Kết quả quan sát phù hợp với mục tiêu đề ra.

5. Tài liệu tham khảo

<https://devtron.ai/blog/create-ci-cd-pipelines-with-github-actions-for-kubernetes-the-definitive-guide/>

<https://kubernetes.io/docs/concepts/services-networking/>

https://www.kevsrobots.com/learn/k3s/14_ci_cd_on_k3s

<https://www.copado.com/resources/blog/kubernetes-load-balancer-strategies-for-maximum-availability-and-scalability>